Python: exercises on functions

This notebook was generated in **pseudocode** mode.

Exercise: Greeting Function

Exercise Description

Create a function `greet` that takes a user's name as input and prints a greeting message. The function should have a default parameter that uses "Fabio" as the default name if no name is provided.

The function should print: "Hello, [name]! Welcome!"

Your solution

```
def greet(name="Fabio"):
    # step 1: create a greeting message by concatenating "Hello, " with the name and
    # step 2: print the greeting message
```

Test your solution

```
# Test the function with default parameter
import io
```

```
import sys
from contextlib import redirect_stdout

# Capture output from greet() with default parameter
f = io.StringIO()
with redirect_stdout(f):
    greet()
output = f.getvalue().strip()
assert output == "Hello, Fabio! Welcome!"
```

Test your solution

```
# Test the function with custom name
f = io.StringIO()
with redirect_stdout(f):
    greet("Alice")
output = f.getvalue().strip()
assert output == "Hello, Alice! Welcome!"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Even or Odd Checker Function

Exercise Description

Write a function `is_even` that takes an integer as input and returns `True` if the number is even, `False` if it's odd.

Your solution

```
def is_even(number):
    # step 1: check if the number modulo 2 equals 0 (meaning it's divisible by 2)
    # step 2: return True if the condition is true, False otherwise
```

Test your solution

```
# Test with even number
assert is_even(4) == True
assert is_even(0) == True
assert is_even(100) == True
```

Test your solution

```
# Test with odd number
assert is_even(3) == False
assert is_even(1) == False
assert is_even(99) == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Maximum of Two Numbers

Exercise Description

Create a function `max_of_two` that returns the larger of two numbers. The second parameter should have a default value of 0.

Your solution

```
def max_of_two(a, b=0):
    # step 1: check if the first number is greater than the second number
        # step 2: return the first number
    # step 3: otherwise
        # step 4: return the second number
```

Test your solution

```
# Test with two positive numbers
assert max_of_two(5, 3) == 5
assert max_of_two(2, 8) == 8
assert max_of_two(10, 10) == 10
```

Test your solution

```
# Test with default parameter
assert max_of_two(5) == 5  # 5 > 0 (default)
```

```
assert max_of_two(-3) == 0  # -3 < 0 (default)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Simple Calculator Function

Exercise Description

Implement a function `calculator` that takes two numbers and an operator (`+`, `-`, `*`, `/`) and returns the result of the operation. Handle division by zero and invalid operators appropriately.

Your solution

```
def calculator(a, b, operator):
    # step 1: check if operator is '+' (addition)
        # step 2: return the sum of a and b
    # step 3: otherwise, check if operator is '-' (subtraction)
        # step 4: return the difference of a and b
    # step 5: otherwise, check if operator is '*' (multiplication)
        # step 6: return the product of a and b
    # step 7: otherwise, check if operator is '/' (division)
        # step 8: check if b is not zero to avoid division by zero
```

```
            # step 9: return the division of a by b
        # step 10: otherwise (division by zero)
            # step 11: return error message "Error: Division by zero"
    # step 12: otherwise (invalid operator)
        # step 13: return error message "Invalid operator"
```

Test your solution

```
# Test basic operations
assert calculator(10, 5, '+') == 15
assert calculator(10, 5, '-') == 5
assert calculator(10, 5, '*') == 50
assert calculator(10, 5, '/') == 2.0
```

Test your solution

```
# Test error cases
assert calculator(10, 0, '/') == "Error: Division by zero"
assert calculator(10, 5, '%') == "Invalid operator"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Temperature Converter Function

Exercise Description

Write a function `celsius_to_fahrenheit` that converts a temperature from Celsius to Fahrenheit using the formula: F = (C × 9/5) + 32

Your solution

```
def celsius_to_fahrenheit(celsius):
    # step 1: apply the conversion formula: multiply celsius by 9/5 and add 32
    # step 2: return the calculated fahrenheit value
```

Test your solution

```
# Test common temperature conversions
assert celsius_to_fahrenheit(0) == 32.0  # Freezing point
assert celsius_to_fahrenheit(100) == 212.0  # Boiling point
assert celsius_to_fahrenheit(25) == 77.0  # Room temperature
```

Test your solution

```
# Test negative temperatures
assert celsius_to_fahrenheit(-10) == 14.0
assert celsius_to_fahrenheit(-40) == -40.0  # Special case where C = F
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Factorial Calculator (Recursive)

Exercise Description

Create a recursive function `factorial` that returns the factorial of a given number. The function should have a default parameter with the value 10. Remember that factorial of n is n! = n × (n-1) × (n-2) × ... × 1, and 0! = 1! = 1.

Your solution

```
def factorial(n=10):
    # step 1: check if n equals 0 or n equals 1 (base cases)
        # step 2: return 1 (since 0! = 1! = 1)
    # step 3: otherwise (recursive case)
        # step 4: return n multiplied by factorial of (n-1)
```

Test your solution

```
# Test base cases
assert factorial(0) == 1
assert factorial(1) == 1
```

Test your solution

```
# Test small factorials
assert factorial(3) == 6  # 3! = 3 × 2 × 1 = 6
```

```
assert factorial(4) == 24  # 4! = 4 × 3 × 2 × 1 = 24
assert factorial(5) == 120  # 5! = 5 × 4 × 3 × 2 × 1 = 120
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Leap Year Checker Function

Exercise Description

Write a function `is_leap_year` that determines whether a given year is a leap year. A leap year is divisible by 4, but if it's divisible by 100, it must also be divisible by 400.

Your solution

```
def is_leap_year(year):
    # step 1: check the complex leap year condition using logical operators
        # step 2: check if year is divisible by 4 AND not divisible by 100
        # step 3: OR check if year is divisible by 400
        # step 4: if either condition is true, return True
    # step 5: otherwise
        # step 6: return False
```

Test your solution

```
# Test leap years
assert is_leap_year(2024) == True  # Divisible by 4, not by 100
assert is_leap_year(2000) == True  # Divisible by 400
assert is_leap_year(1600) == True  # Divisible by 400
```

Test your solution

```
# Test non-leap years
assert is_leap_year(2023) == False  # Not divisible by 4
assert is_leap_year(1900) == False  # Divisible by 100 but not by 400
assert is_leap_year(2100) == False  # Divisible by 100 but not by 400
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Circle Area Calculator

Exercise Description

Implement a function `circle_area` that calculates the area of a circle given its radius. Use the formula: Area = π × radius²

Your solution

```
import math

def circle_area(radius):
    # step 1: calculate the area using the formula: pi times radius squared
    # step 2: return the calculated area
```

Test your solution

```
# Test with common radius values
import math
assert abs(circle_area(1) - math.pi) < 0.0001  # Area of unit circle
assert abs(circle_area(2) - 4 * math.pi) < 0.0001  # Area = 4π
```

Test your solution

```
# Test with larger radius
assert abs(circle_area(5) - 25 * math.pi) < 0.0001  # Area = 25π
assert circle_area(0) == 0  # Zero radius
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Vowel Checker Function

Exercise Description

Create a function `is_vowel` that checks whether a given character is a vowel (a, e, i, o, u) in either uppercase or lowercase.

Your solution

```
def is_vowel(char):
    # step 1: create a string containing all vowels in both lowercase and uppercase
    # step 2: check if the given character is in the vowels string
    # step 3: return True if found, False otherwise
```

Test your solution

```
# Test with lowercase vowels
assert is_vowel('a') == True
assert is_vowel('e') == True
assert is_vowel('i') == True
assert is_vowel('o') == True
assert is_vowel('u') == True
```

Test your solution

```
# Test with uppercase vowels and consonants
assert is_vowel('A') == True
assert is_vowel('E') == True
assert is_vowel('b') == False
assert is_vowel('Z') == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Simple Interest Calculator

Exercise Description

Write a function `simple_interest` that calculates simple interest given principal amount, rate of interest, and time period. Use the formula: Simple Interest = (Principal × Rate × Time) / 100

Your solution

```
def simple_interest(principal, rate, time):
    # step 1: calculate simple interest using the formula: (principal * rate * time)
    # step 2: return the calculated interest amount
```

Test your solution

```
# Test with common values
assert simple_interest(1000, 5, 2) == 100.0  # $1000 at 5% for 2 years
assert simple_interest(5000, 10, 1) == 500.0  # $5000 at 10% for 1 year
```

Test your solution

```
# Test with different scenarios
assert simple_interest(2000, 7.5, 3) == 450.0  # $2000 at 7.5% for 3 years
assert simple_interest(1000, 0, 5) == 0.0  # 0% interest rate
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Count Vowels Function

Exercise Description

Write a function `count_vowels` that counts the number of vowels (a, e, i, o, u) in a given string, ignoring case.

Your solution

```
def count_vowels(text):
    # step 1: define a string containing all vowels in both cases
    # step 2: initialize a counter to 0
    # step 3: iterate through each character in the text
        # step 4: check if the character is in the vowels string
```

```
        # step 5: if yes, increment the counter
    # step 6: return the total count
```

Test your solution

```
# Test with various strings
assert count_vowels("hello") == 2  # e, o
assert count_vowels("Python") == 1  # o
assert count_vowels("aeiou") == 5  # All vowels
```

Test your solution

```
# Test with mixed case and edge cases
assert count_vowels("HELLO") == 2  # E, O
assert count_vowels("bcdfg") == 0  # No vowels
assert count_vowels("") == 0  # Empty string
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Absolute Value Function

Exercise Description

Create a function `absolute` that returns the absolute value of a number (the non-negative value without regard to its sign).

Your solution

```
def absolute(num):
    # step 1: check if the number is greater than or equal to 0
        # step 2: return the number as is (it's already positive)
    # step 3: otherwise (the number is negative)
        # step 4: return the negative of the number (making it positive)
```

Test your solution

```
# Test with positive numbers
assert absolute(5) == 5
assert absolute(10.5) == 10.5
assert absolute(0) == 0
```

Test your solution

```
# Test with negative numbers
assert absolute(-5) == 5
assert absolute(-10.5) == 10.5
assert absolute(-100) == 100
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Maximum of Three Numbers

Exercise Description

Write a function `max_of_three` that returns the largest of three numbers.

Your solution

```
def max_of_three(a, b, c):
    # step 1: check if a is greater than or equal to both b and c
        # step 2: return a as the maximum
    # step 3: otherwise, check if b is greater than or equal to both a and c
        # step 4: return b as the maximum
    # step 5: otherwise (c must be the maximum)
        # step 6: return c as the maximum
```

Test your solution

```
# Test with different maximum positions
assert max_of_three(10, 5, 8) == 10  # First is maximum
assert max_of_three(3, 15, 7) == 15  # Second is maximum
assert max_of_three(4, 6, 20) == 20  # Third is maximum
```

Test your solution

```
# Test with equal values
assert max_of_three(5, 5, 5) == 5  # All equal
```

```
assert max_of_three(10, 10, 8) == 10  # Two equal maximums
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Power Function

Exercise Description

Write a function `power` that calculates the power of a number (base raised to the exponent).

Your solution

```
def power(base, exponent):
    # step 1: calculate base raised to the power of exponent using the ** operator
    # step 2: return the calculated result
```

Test your solution

```
# Test with common power calculations
assert power(2, 3) == 8  # 2^3 = 8
assert power(5, 2) == 25  # 5^2 = 25
assert power(10, 0) == 1  # Any number to power 0 is 1
```

Test your solution

```
# Test with edge cases
assert power(3, 1) == 3  # Any number to power 1 is itself
assert power(2, 4) == 16  # 2^4 = 16
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Palindrome Checker Function

Exercise Description

Write a function `is_palindrome` that checks whether a given string reads the same forwards and backwards.

Your solution

```
def is_palindrome(text):
    # step 1: convert the text to lowercase for case-insensitive comparison
    # step 2: compare the text with its reverse
    # step 3: return True if they are equal, False otherwise
```

Test your solution

```
# Test with palindromes
assert is_palindrome("racecar") == True
assert is_palindrome("level") == True
assert is_palindrome("a") == True  # Single character
```

Test your solution

```
# Test with non-palindromes and case variations
assert is_palindrome("hello") == False
assert is_palindrome("Racecar") == True  # Case insensitive
assert is_palindrome("python") == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Calculate Body Mass Index (BMI)

Exercise Description

Write a function `calculate_bmi(weight, height)` that returns the Body Mass Index given weight (in kilograms) and height (in meters). The BMI is computed as `weight /`

`(height**2)`.

Your solution

```
def calculate_bmi(weight, height):
    # compute the body mass index by dividing the weight by the square of the height
    # return the computed bmi value
```

Test your solution

```
# typical values
assert round(calculate_bmi(70, 1.75), 2) == 22.86
assert round(calculate_bmi(60, 1.6), 2) == 23.44
```

Test your solution

```
# edge-like scenarios
assert round(calculate_bmi(90, 2.0), 2) == 22.5
assert round(calculate_bmi(45, 1.5), 2) == 20.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Convert Minutes to Hours and Minutes

Exercise Description

Implement a function `convert_minutes(total_minutes)` that takes a non-negative integer number of minutes and returns a tuple `(hours, minutes)` representing the equivalent hours and remaining minutes.

Your solution

```
def convert_minutes(total_minutes):
    # compute how many whole hours are contained in the total minutes using integer d
    # compute the remaining minutes using the remainder operator with 60
    # return the pair (hours, minutes) as a tuple
```

Test your solution

```
# basic cases
assert convert_minutes(0) == (0, 0)
assert convert_minutes(59) == (0, 59)
assert convert_minutes(60) == (1, 0)
```

Test your solution

```
# additional values
assert convert_minutes(125) == (2, 5)
assert convert_minutes(240) == (4, 0)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Minimum of Two Numbers

Exercise Description

Create a function `min_of_two(a, b)` that returns the smaller of two numbers `a` and `b`.

Your solution

```
def min_of_two(a, b):
    # check if a is less than b
        # if true, return a because it is the smaller value
    # otherwise
        # return b because it is less than or equal to a
```

Test your solution

```
# typical comparisons
assert min_of_two(3, 5) == 3
assert min_of_two(-1, 2) == -1
assert min_of_two(7, 7) == 7
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Determine Grade Based on Score

Exercise Description

Write a function `determine_grade(score)` that returns a letter grade based on a numeric score from 0 to 100 inclusive. Use the following scale: A for 90+, B for 80–89, C for 70–79, D for 60–69, F for below 60.

Your solution

```
def determine_grade(score):
    # if score is greater than or equal to 90 then return 'A'
    # else if score is greater than or equal to 80 then return 'B'
    # else if score is greater than or equal to 70 then return 'C'
    # else if score is greater than or equal to 60 then return 'D'
    # otherwise return 'F'
```

Test your solution

```
# boundary checks
assert determine_grade(100) == 'A'
assert determine_grade(90) == 'A'
assert determine_grade(89) == 'B'
assert determine_grade(80) == 'B'
assert determine_grade(79) == 'C'
assert determine_grade(70) == 'C'
assert determine_grade(69) == 'D'
assert determine_grade(60) == 'D'
assert determine_grade(59) == 'F'
assert determine_grade(0) == 'F'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Convert Fahrenheit to Celsius

Exercise Description

Implement a function `fahrenheit_to_celsius(fahrenheit)` that converts a temperature in Fahrenheit to Celsius using the formula `C = (F - 32) * 5 / 9`. Return the Celsius value as a float.

Your solution

```
def fahrenheit_to_celsius(fahrenheit):
    # subtract 32 from the fahrenheit temperature
    # multiply the result by 5
    # divide by 9 to obtain the temperature in celsius
    # return the computed celsius value
```

Test your solution

```
# known reference points
assert round(fahrenheit_to_celsius(32), 2) == 0.00
assert round(fahrenheit_to_celsius(212), 2) == 100.00
assert round(fahrenheit_to_celsius(68), 2) == 20.00
```

Test your solution

```
# additional checks
assert round(fahrenheit_to_celsius(14), 2) == -10.00
assert round(fahrenheit_to_celsius(41), 2) == 5.00
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```